End-to-End CI/CD DevOps : Orchestration d'un Cluster Kubernetes Self-Hosted sur Azure via Terraform & Ansible

Introduction du projet

Context et objectif:

Dans le paysage technologique actuel, la conteneurisation et l'orchestration sont devenues les piliers de la scalabilité applicative. Ce projet, intitulé"CI/CD Devops with self hosted Kubernetes Cluster", a pour objectif de concevoir et de déployer une infrastructure complète et automatisée sur Microsoft Azure pour héberger une application web Angular de manière résiliente.

Problématique Technique:

Contrairement aux solutions managées (Azure AKS) qui masquent la complexité de l'orchestrateur, ce projet repose sur une approche "Self-hosted". Ce choix technique impose de maîtriser l'intégralité de la pile technologique; du provisionnement des ressources matérielles à la configuration fine du plan de contrôle (Control Plane) et des nœuds de travail (workers).

Architecture et Méthodologie:

La réalisation s'appuie sur une démarche Infrastructure-as-code (IaaC) et une chaîne de déploiement continu structuré en trois piliers:

1. **Automatisation:** utilisation de **Terraform** pour le déploiement des ressources Azure (VMs, Réseaux,, Sécurité) et **Ansible** pour la configuration logicielle du cluster.
2. **Pipeline CI/CD:** Intégration d'**Azure Devops** pour automatiser le cycle de vie de l'application, du build de l'image docker jusqu'au déploiement sur le cluster.
3. **Observabilité:** Mise en place d'une stack de monitoring Prometheus & Grafana pour garantir la haute disponibilité et la visibilité des performances.

Structure technique du projet et configuration

Etape 1: Pipeline CI/CD Azure Devops

Conception de la pipeline:

On suit une approche “ALL in One”. La pipeline fusionne la partie CI et CD dans un seul job pour garantir une livraison rapide.

Les blocs de bases:

Trigger: En écoute de la branche master. Dès qu’un nouveau git push, la pipeline se déclenche automatiquement.

Pool: On utilise name: Default. Self hosted agent pour exécuter les commandes.

Les variables:

imageName: Le nom de ton image sur Docker Hub.

serviceConnection: Ce sont les clés" de sécurité. Une pour **Docker Hub** (pousser l'image) et une pour **Kubernetes** (piloter le cluster)

Etape 1: Build & Push (CI)

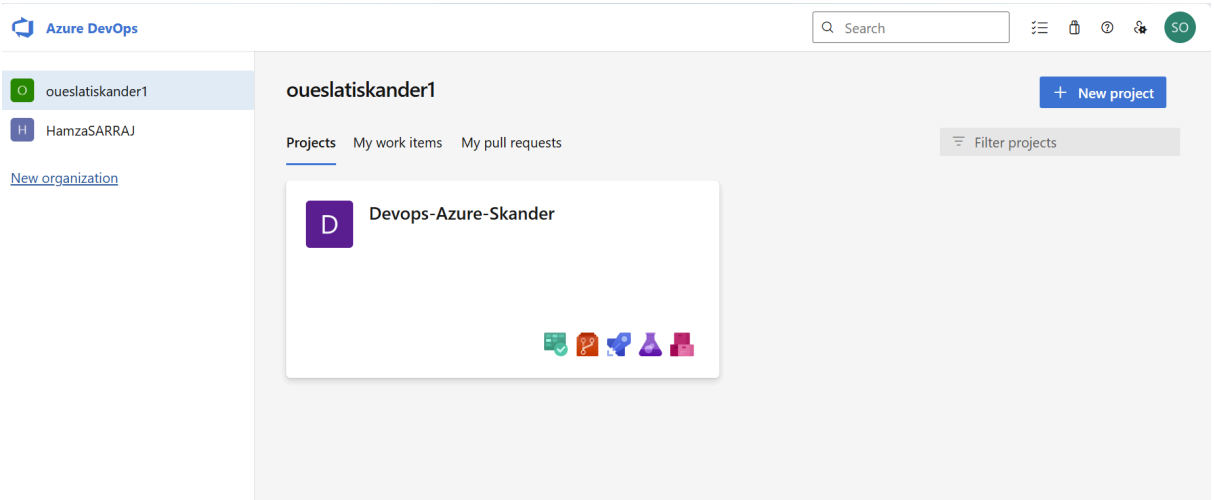
- Tâche: Docker@2
- Action: Elle lit Dockerfile, compile l'application Angular (via les étapes à l'intérieur du Dockerfile) et crée une image.
- Tagging: Création de deux versions; une avec le numéro du build (BuildId) pour la traçabilité et une version latest.
- Destination: L’image envoyée vers DockerHub.

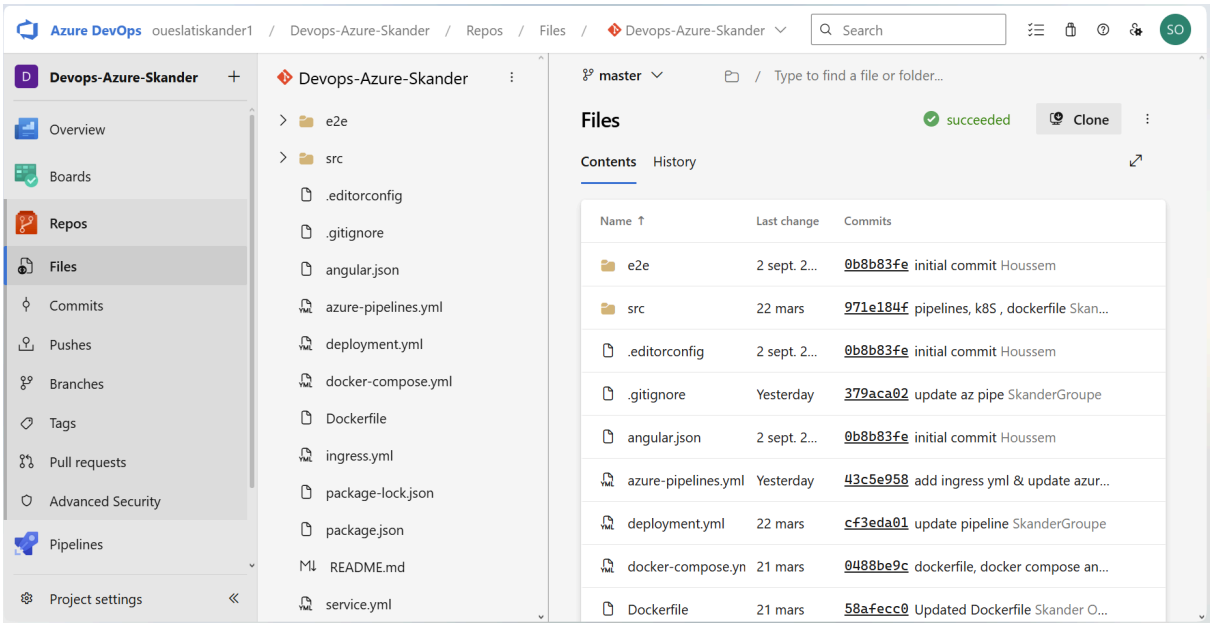
Etape 2: Deploy (CD)

- Tâche: KubernetesManifest@0
- Action: l'envoi des fichiers; deployment.yml, service.yml au cluster
- Intelligence: La ligne containers: ‘$(imageName):$(Build.BuildId): remplace automatiquement l’image dans le YAML file par celle qu’on vient juste de builder.
- Namespace: Elle déploie qa et le crée s’il n’existe pas.

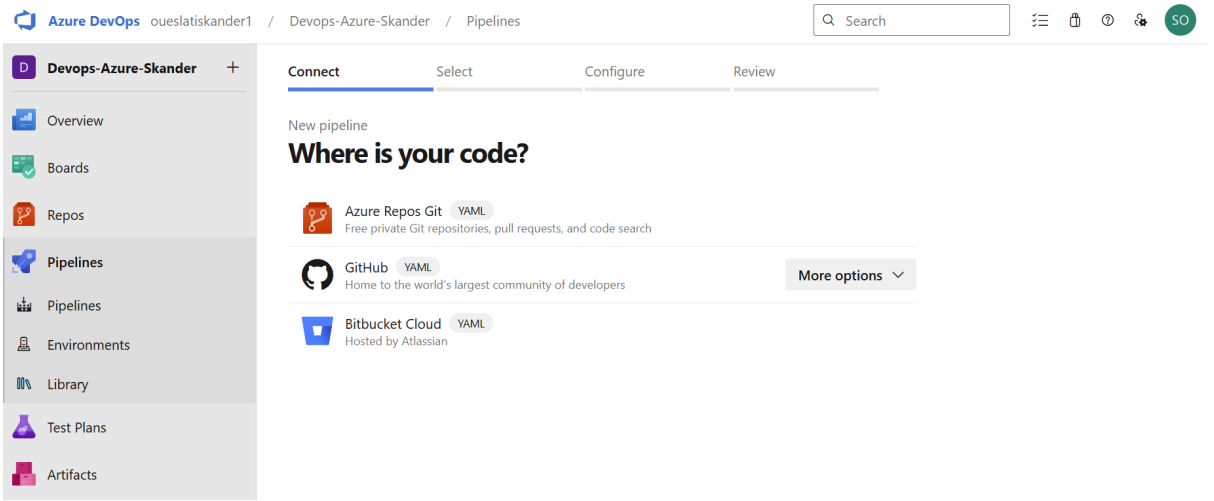
Réalisation:

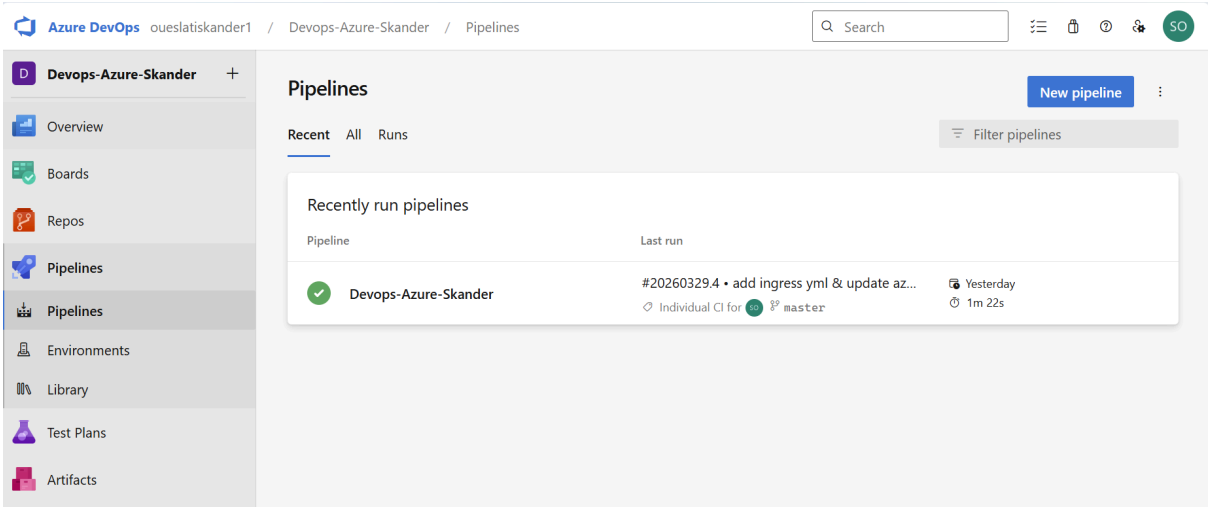
- Upload le projet dans le repo Azure Devops



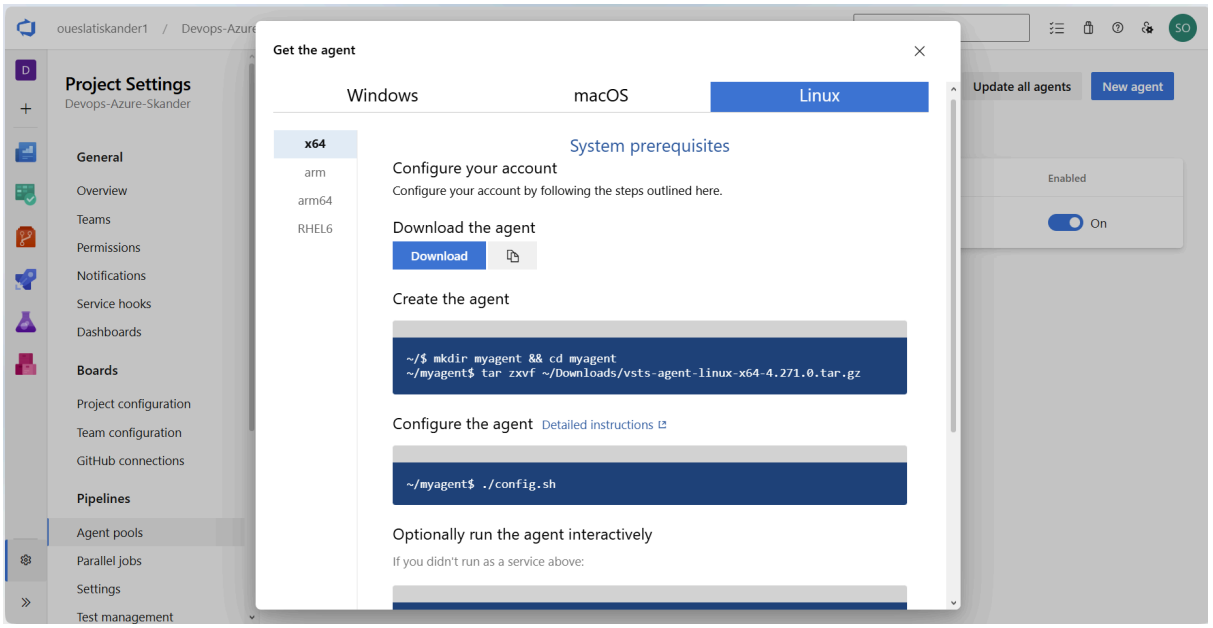


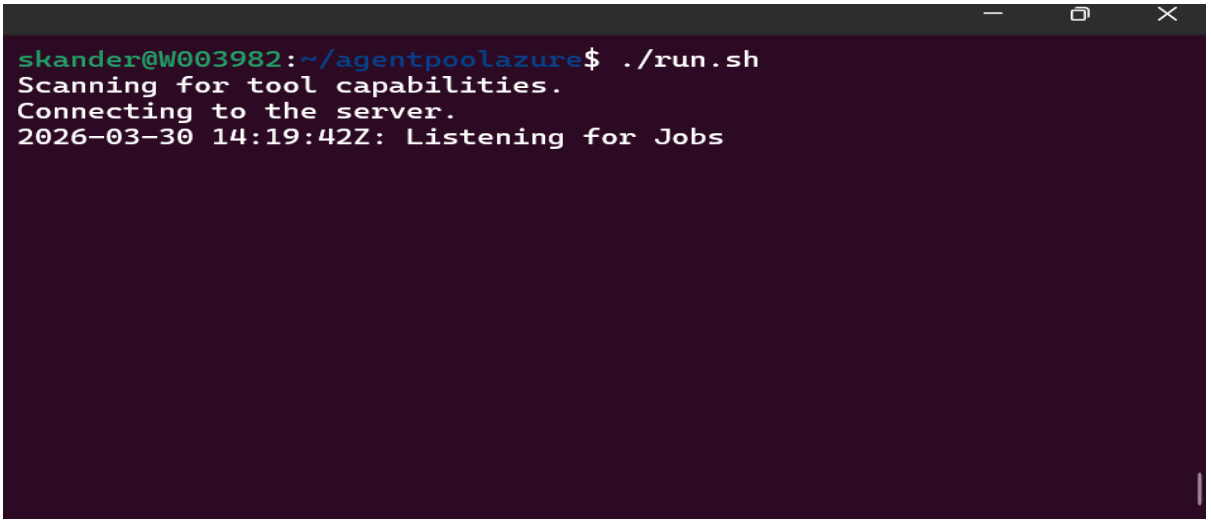
- Création d'une nouvelle pipeline



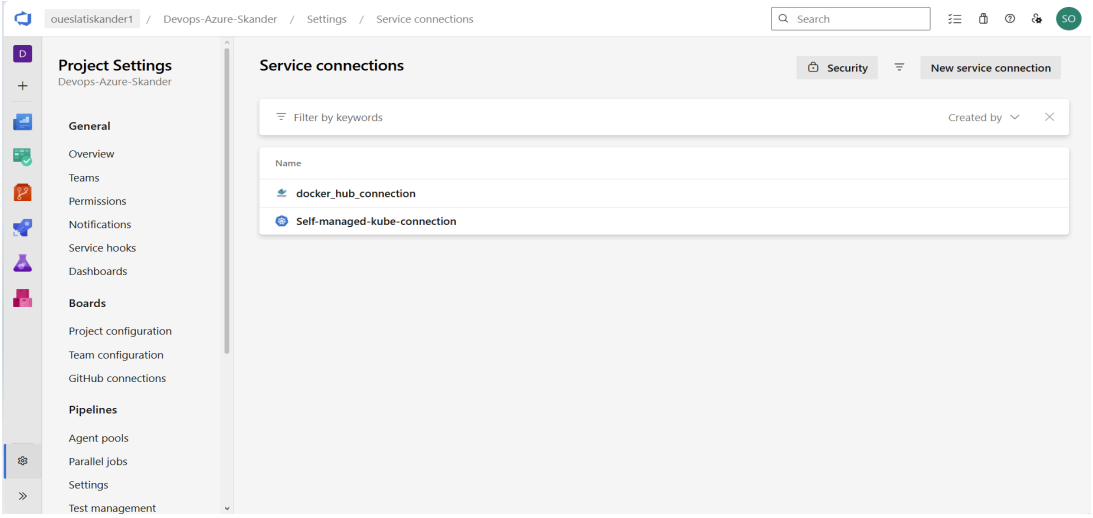


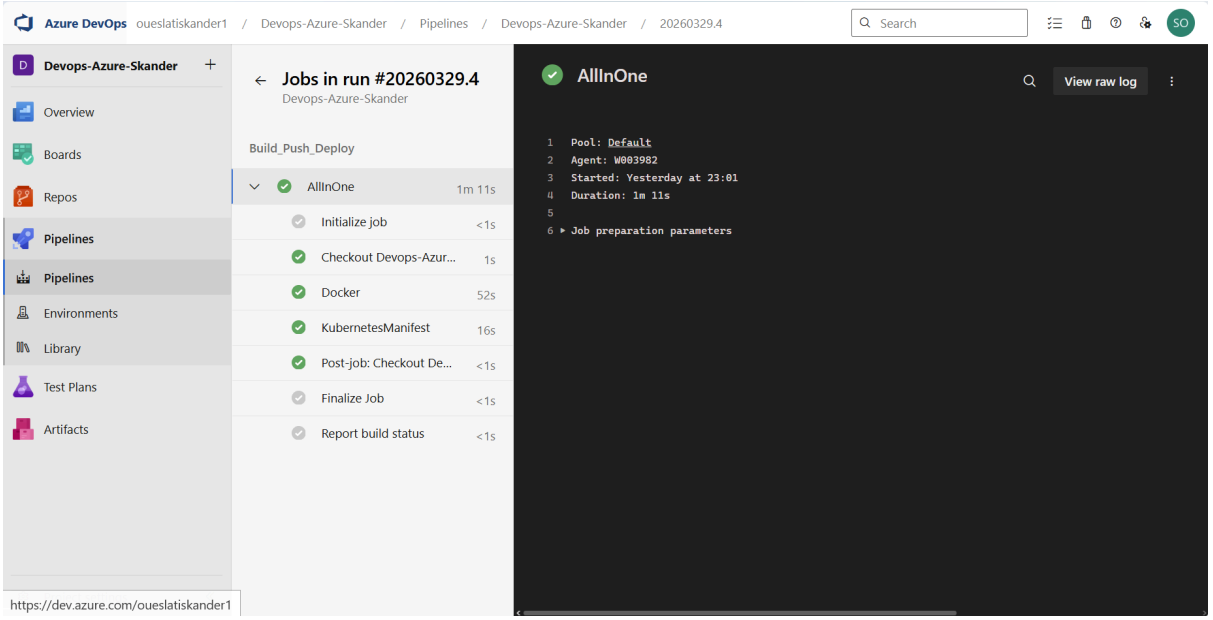
- Création de l'agent pool self managed (sur ma VM local)
  Project Settings -> Agent pools -> Default -> New Agent





- Création des services de connection





Etape 2: Provisionnement de l’infrastructure (Terraform)

L'objectif est de déployer un environnement Azure complet, sécurisé et prêt à accueillir un cluster Kubernetes. L'utilisation du provider azurerm permet de piloter l'API Azure pour créer des ressources de manière déclarative.

Analyse de la configuration (Architecture Réseau):
Le réseau est segmenté pour garantir l'isolement du cluster :

- VNet & Subnet : Création d'un réseau virtuel (10.0.0.0/16) et d'un sous-réseau spécifique (10.0.1.0/24) pour le cluster.
- Adressage IP : Utilisation d'adresses IP Publiques Statiques de SKU "Standard" pour garantir que les adresses ne changent pas lors d'un redémarrage, ce qui est crucial pour la stabilité de l'API Kubernetes.

Sécurité Périmétrique (Network Security Group):

Le fichier main.tf définit un pare-feu (NSG) strict avec des règles spécifiques pour le fonctionnement de Kubernetes :

- Port 22 (SSH) : Pour l'administration via Ansible.
- Port 6443 (K8s-API) : Indispensable pour que kubectl et les composants du cluster communiquent avec l'API Server.
- Port 10250 (Kubelet) : Nécessaire pour la gestion des nœuds.
- Port 4789 (Calico/UDP) : Permet l'encapsulation VXLAN pour le réseau entre les Pods.
- Plage 30000-32767 (NodePort) : Ouvre l'accès extérieur à ton application Angular et à Grafana.

Optimisation via les locals et for_each:

Au lieu de dupliquer le code pour le Master et le Worker, tu as utilisé un bloc locals :
Terraform
locals {
  nodes = {

```
    "master" = { size = "Standard_D2s_v3", id = 1 }
    "worker" = { size = "Standard_D2s_v3", id = 2 }
  }
}
```

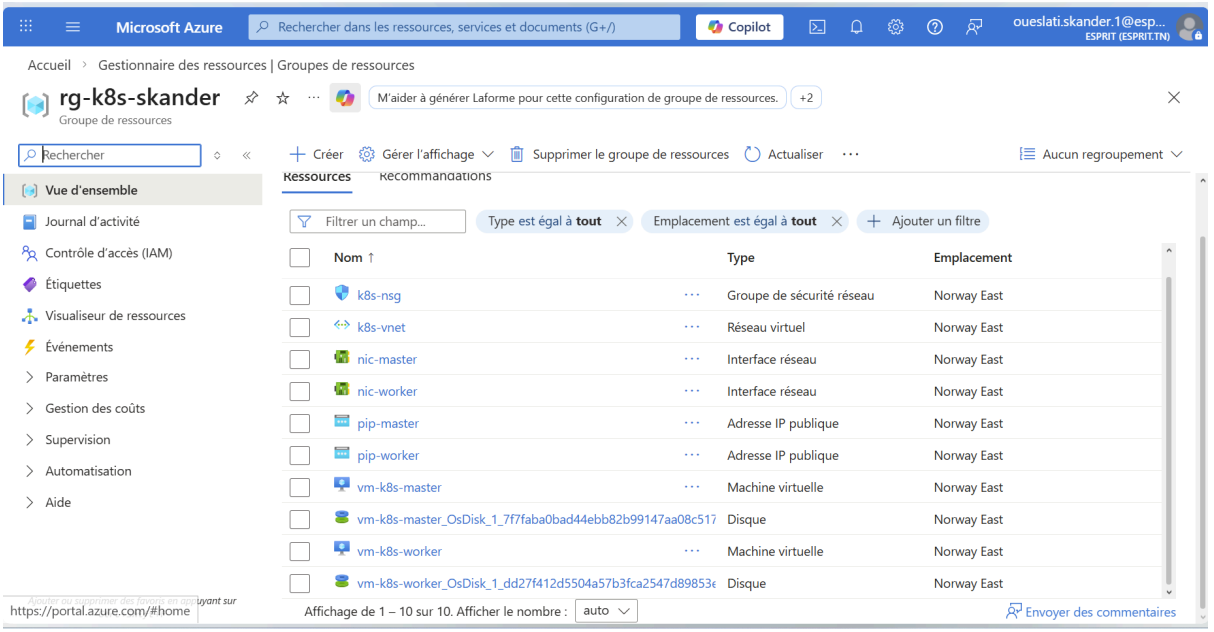
Cette approche permet de créer dynamiquement les interfaces réseau, les IPs publiques et les VMs. C'est une "Best Practice" Terraform car elle facilite l'ajout de nouveaux nœuds workers simplement en modifiant une ligne.

Sorties (Outputs):
Les outputs permettent d'afficher directement dans ton terminal les adresses IP publiques générées par Azure. Ces IPs sont ensuite utilisées pour configurer la Service Connection dans Azure DevOps.

Réalisation:

- Execution de la commande Terraform init - terraform plan - terraform apply
  Lien github du terraform files: [terraform files]

```
Apply complete! Resources: 12 added, 0 changed, 0 destroyed.

Outputs:

master_public_ip = "20.100.190.29"
worker_public_ip = "20.100.178.105"
skander@W003982:~/angular-app-kubernetes-terraform$
```



Etape 3: Configuration du Cluster

Prep **worker+master**

```shell
# Désactiver le swap immédiatement et au redémarrage
sudo swapoff -a
sudo sed -i '/ swap / s/^\(.*\)$/#\1/g' /etc/fstab

# Charger les modules noyau nécessaires
cat <<EOF | sudo tee /etc/modules-load.d/k8s.conf
overlay
br_netfilter
EOF

sudo modprobe overlay
sudo modprobe br_netfilter

# Configurer les paramètres sysctl pour le réseau (IP Forwarding)
cat <<EOF | sudo tee /etc/sysctl.d/k8s.conf
net.bridge.bridge-nf-call-iptables  = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.ip_forward                 = 1
EOF

# Appliquer les paramètres sans redémarrer
sudo sysctl --system
```

Install containerD **worker+master**

```shell
sudo apt-get update
sudo apt-get install -y containerd

# Générer la configuration par défaut et activer SystemdCgroup
sudo mkdir -p /etc/containerd
containerd config default | sudo tee /etc/containerd/config.toml > /dev/null
sudo sed -i 's/SystemdCgroup = false/SystemdCgroup = true/g'
/etc/containerd/config.toml

sudo systemctl restart containerd
```

Install Kubeadm,kubectl kubelet **wroker+master**

```shell
sudo apt-get update
sudo apt-get install -y apt-transport-https ca-certificates curl gpg
```

```
# Ajouter la clé GPG et le dépôt (version v1.29 par exemple)
curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.29/deb/Release.key | sudo
gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg

echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg]
https://pkgs.k8s.io/core:/stable:/v1.29/deb/ /' | sudo tee
/etc/apt/sources.list.d/kubernetes.list

sudo apt-get update
sudo apt-get install -y kubelet kubeadm kubectl
sudo apt-mark hold kubelet kubeadm kubectl
```

Initiat **master**

```Shell
# Remplace 10.244.0.0/16 par le réseau de ton choix si besoin (standard pour
Flannel)
sudo kubeadm init --pod-network-cidr=10.244.0.0/16

# Pour pouvoir utiliser kubectl en tant qu'utilisateur normal :
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

Installation réseau pod **master**:

```Shell
kubectl apply -f
https://raw.githubusercontent.com/flannel-io/flannel/master/Documentation/ku
be-flannel.yml
```

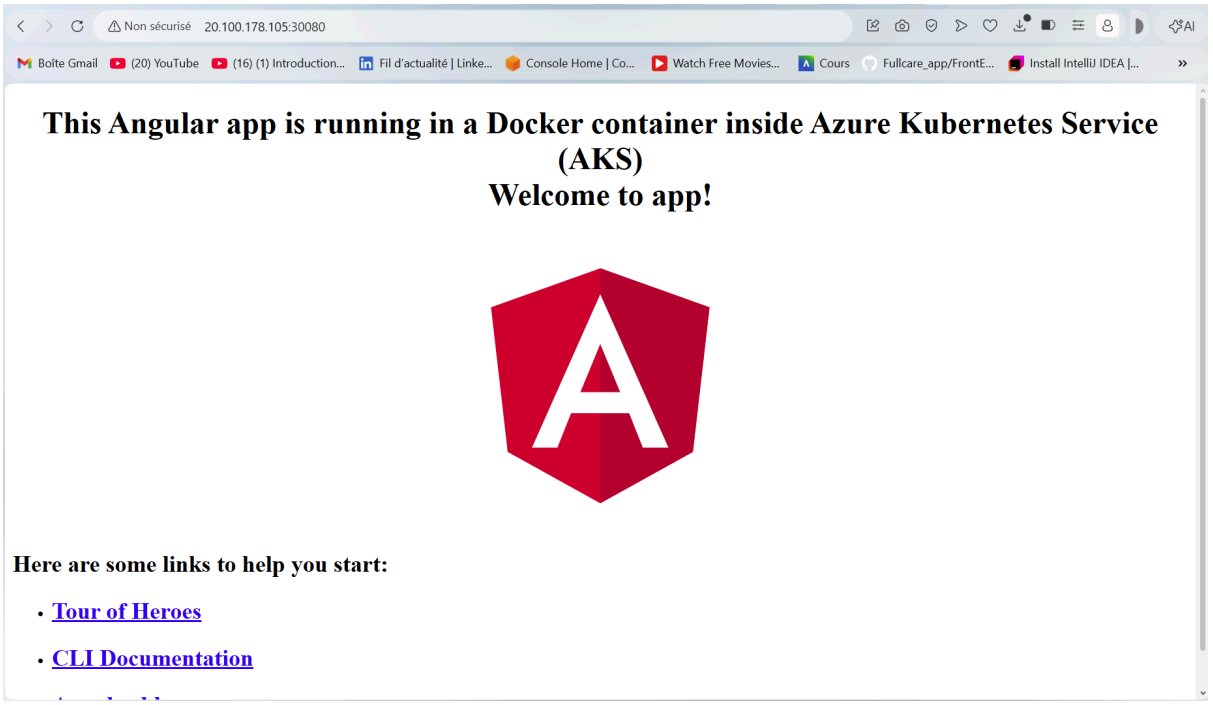
Join **worker**

```Shell

sudo kubeadm join <IP-DU-MASTER>:6443 --token <votre-token>
--discovery-token-ca-cert-hash sha256:<votre-hash>
```

```
13 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

98 additional security updates can be applied with ESM Infra.
Learn more about enabling ESM Infra service for Ubuntu 20.04 at
https://ubuntu.com/20-04

New release '22.04.5 LTS' available.
Run 'do-release-upgrade' to upgrade to it.


*** System restart required ***
Last login: Mon Mar 30 17:46:05 2026 from 197.2.75.196
skander@vm-k8s-master:~$ kubectl get nodes
NAME            STATUS   ROLES           AGE   VERSION
vm-k8s-master   Ready    control-plane   27h   v1.29.15
vm-k8s-worker   Ready    <none>          27h   v1.29.15
skander@vm-k8s-master:~$
```

```
MicroK8s
   just raised the bar for easy, resilient and secure K8s cluster dep
loyment.

   https://ubuntu.com/engage/secure-kubernetes-at-the-edge

Expanded Security Maintenance for Infrastructure is not enabled.

34 updates can be applied immediately.
21 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

98 additional security updates can be applied with ESM Infra.
Learn more about enabling ESM Infra service for Ubuntu 20.04 at
https://ubuntu.com/20-04


Last login: Sun Mar 29 20:33:46 2026 from 102.31.182.86
skander@vm-k8s-worker:~$
```

Etape 4: Exposition du service NodePort http://<ip_public>:30080



Etape 6: Observabilité (Prometheus & Grafana)

Pour surveiller à la fois les ressources des machines virtuelles (VM) et l'application Kubernetes, l'approche recommandée consiste à déployer une pile de monitoring complète à l'intérieur du cluster :

- Prometheus → Collecte des métriques.
- Grafana → Visualisation des données (Dashboards).
- Node Exporter → Métriques des VMs (CPU, RAM, Disque).
- kube-state-metrics → État des objets Kubernetes (Pods, Deployments).

La méthode la plus simple et prête pour la production : Helm + kube-prometheus-stack.

Install Helm (on master node):

```shell
curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash
```

```
helm version
```

Add Prometheus community repo:

```shell
helm repo add prometheus-community
https://prometheus-community.github.io/helm-charts
helm repo update
```

Déployé le monitoring stack:

```shell
kubectl create namespace monitoring

helm install monitoring
prometheus-community/kube-prometheus-stack \
  --namespace monitoring
```

- Prometheus
- Grafana
- Alertmanager
- Node Exporter (VM metrics)
- kube-state-metrics

Expose Grafana (important):

By default, Grafana is ClusterIP → not accessible.

```shell
kubectl patch svc monitoring-grafana -n monitoring \
  -p '{"spec": {"type": "NodePort"}}'
```

Get port:

```shell
kubectl get svc -n monitoring
```

Access:

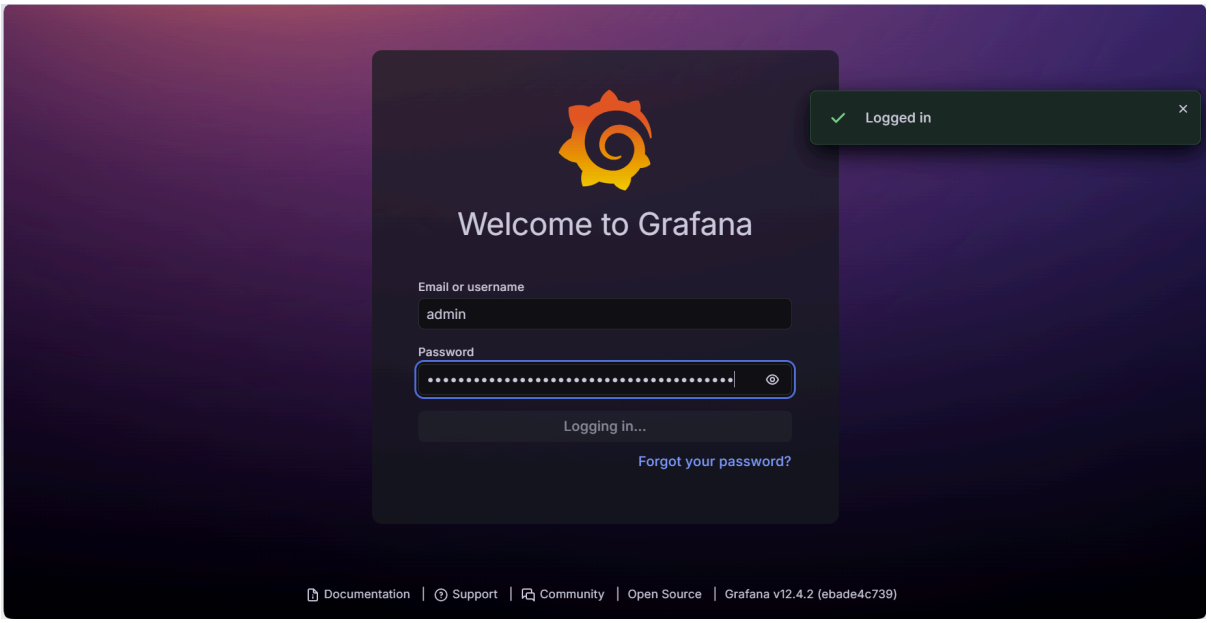
```
None
http://<VM_PUBLIC_IP>:<NODE_PORT>
```

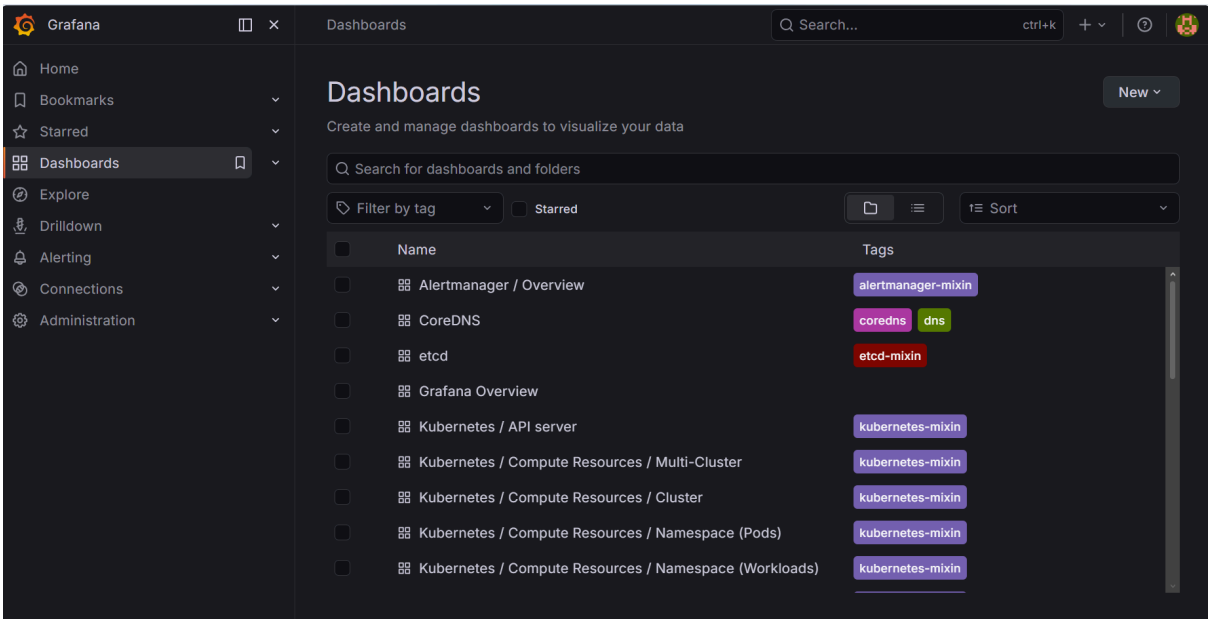
Get Grafana admin password:

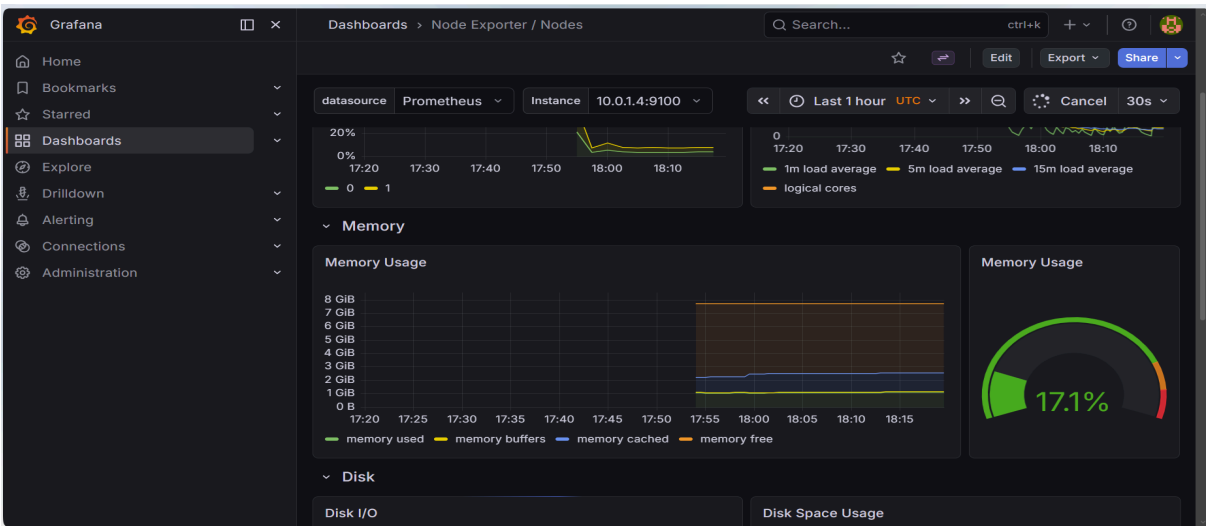
```shell
kubectl get secret monitoring-grafana -n monitoring \
  -o jsonpath="{.data.admin-password}" | base64 --decode
```

Login:

- user: `admin`
- password: (command above)


Logged in
Welcome to Grafana
Email or username
admin
Password
Logging in...
Forgot your password?
Documentation | Support | Community | Open Source | Grafana v12.4.2 (ebade4c739)


Grafana
Dashboards
Search...
ctrl+k
Home
Bookmarks
Starred
Dashboards
Explore
Drilldown
Alerting
Connections
Administration
Dashboards
New
Create and manage dashboards to visualize your data
Search for dashboards and folders
Filter by tag
Starred
Sort
Name
Tags
Alertmanager / Overview
alertmanager-mixin
CoreDNS
coredns
dns
etcd
etcd-mixin
Grafana Overview
Kubernetes / API server
kubernetes-mixin
Kubernetes / Compute Resources / Multi-Cluster
kubernetes-mixin
Kubernetes / Compute Resources / Cluster
kubernetes-mixin
Kubernetes / Compute Resources / Namespace (Pods)
kubernetes-mixin
Kubernetes / Compute Resources / Namespace (Workloads)
kubernetes-mixin


Grafana
Dashboards > Node Exporter / Nodes
Search...
ctrl+k
Edit
Export
Share
datasource
Prometheus
Instance
10.0.1.4:9100
Last 1 hour
UTC
Cancel
30s
20%
0%
17:20
17:30
17:40
17:50
18:00
18:10
0
1
1m load average
5m load average
15m load average
logical cores
Memory
Memory Usage
8 GiB
7 GiB
6 GiB
5 GiB
4 GiB
3 GiB
2 GiB
1 GiB
0 B
17:20
17:25
17:30
17:35
17:40
17:45
17:50
17:55
18:00
18:05
18:10
18:15
memory used
memory buffers
memory cached
memory free
Memory Usage
17.1%
Disk
Disk I/O
Disk Space Usage